

Master Informatique
Princip De Programmation

Group Trip Planner

Développement & Déploiement d'une Application Web RESTful
Conteneurisée

Zeynab SALEHI

Boboc TEODORA

Mathurin MARTEL

Encadré par : M. Samir YUCEF

Le défi de l'organisation collective

Un véritable casse-tête

Organiser un voyage à plusieurs est souvent synonyme de chaos logistique :

-  **Dispersion** Informations perdues entre les mails et les messages.
-  **Fragmentation** Notes éparpillées sur plusieurs supports.
-  **Complexité** Une planification laborieuse et peu fluide.



Une plateforme collaborative complète



Itinéraires

Centralisation des étapes,
des hôtels et des activités en
un seul lieu.



Participants

Accès partagé au programme
en temps réel pour tous les
membres.



Budget

Suivi précis des dépenses
prévues pour une
transparence totale.

"Simplifier l'organisation de A à Z."

Architecture & Infrastructure

-  **Backend** : Python & FastAPI (Rapidité & Documentation automatique).
-  **Données** : PostgreSQL avec l'ORM SQLAlchemy.
-  **Déploiement** : Conteneurisation totale via Docker.
-  **Orchestration** : Docker Compose (Backend + DB).

```
app/ # Logique backend
├── crud.py # Logique métier
├── database.py # Session PostgreSQL
├── main.py # Point d'entrée & CORS
├── models.py # Structure SQLAlchemy
├── routes_b.py # Endpoints
├── schemas.py # Validation Pydantic
└── frontend/ # Interface web
    ├── index.html # Structure de la page
    ├── styles.css # Design
    └── app.js # Logique Fetch/Async
Dockerfile # Image de l'app
docker-compose.yml # Orchestration
init.sql # Schéma initial
```

Structure du

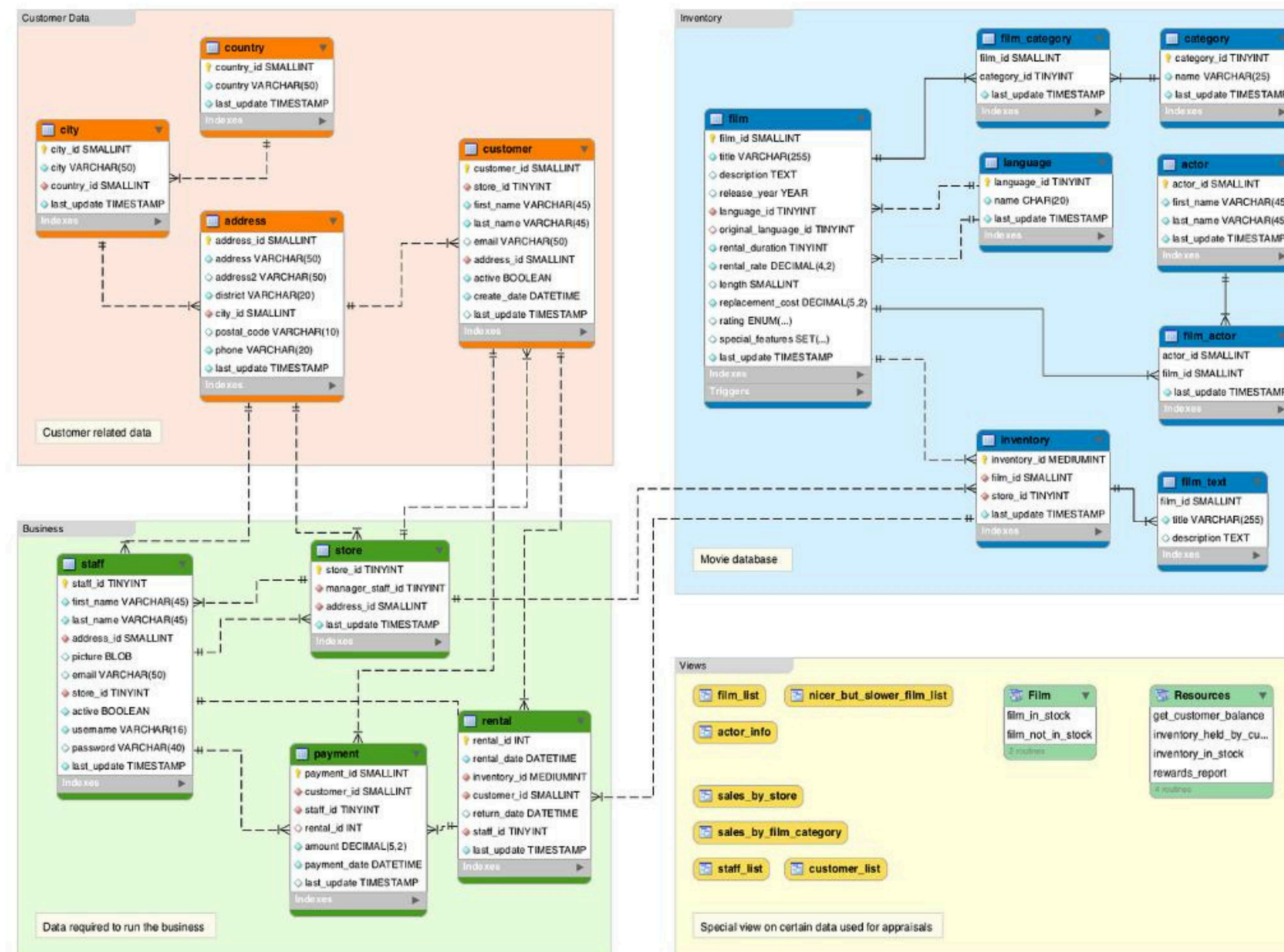
Frontend : Simplicité & Performance

```
// Zero Framework Strategy
- HTML5 Sémantique (, )
- CSS3 Moderne (Flexbox, Grid, Variables)
- JavaScript Vanilla (ES6+)
- API Fetch & Async/Await
- LocalStorage (Persistance URL API)
```

Zéro dépendance npm

-  **Légèreté** : Projet structuré en trois fichiers (HTML, CSS, JS).
-  **Typographie** : Intégration Google Fonts (Fraunces, Inter).
-  **Réactivité** : Communication fluide avec l'API sans build step.

Modélisation & SQLAlchemy



Intégrité & Abstraction

Utilisation de SQLAlchemy pour manipuler les données comme des objets Python :

-  **Many-to-Many :** Partage d'un voyage entre plusieurs utilisateurs.
-  **One-to-Many :** Liste des destinations par voyage.
-  **One-to-One :** Budget unique par voyage.

Une organisation professionnelle



models.py

Structure de la base de données via SQLAlchemy.

schemas.py

Validation des données avec Pydantic.



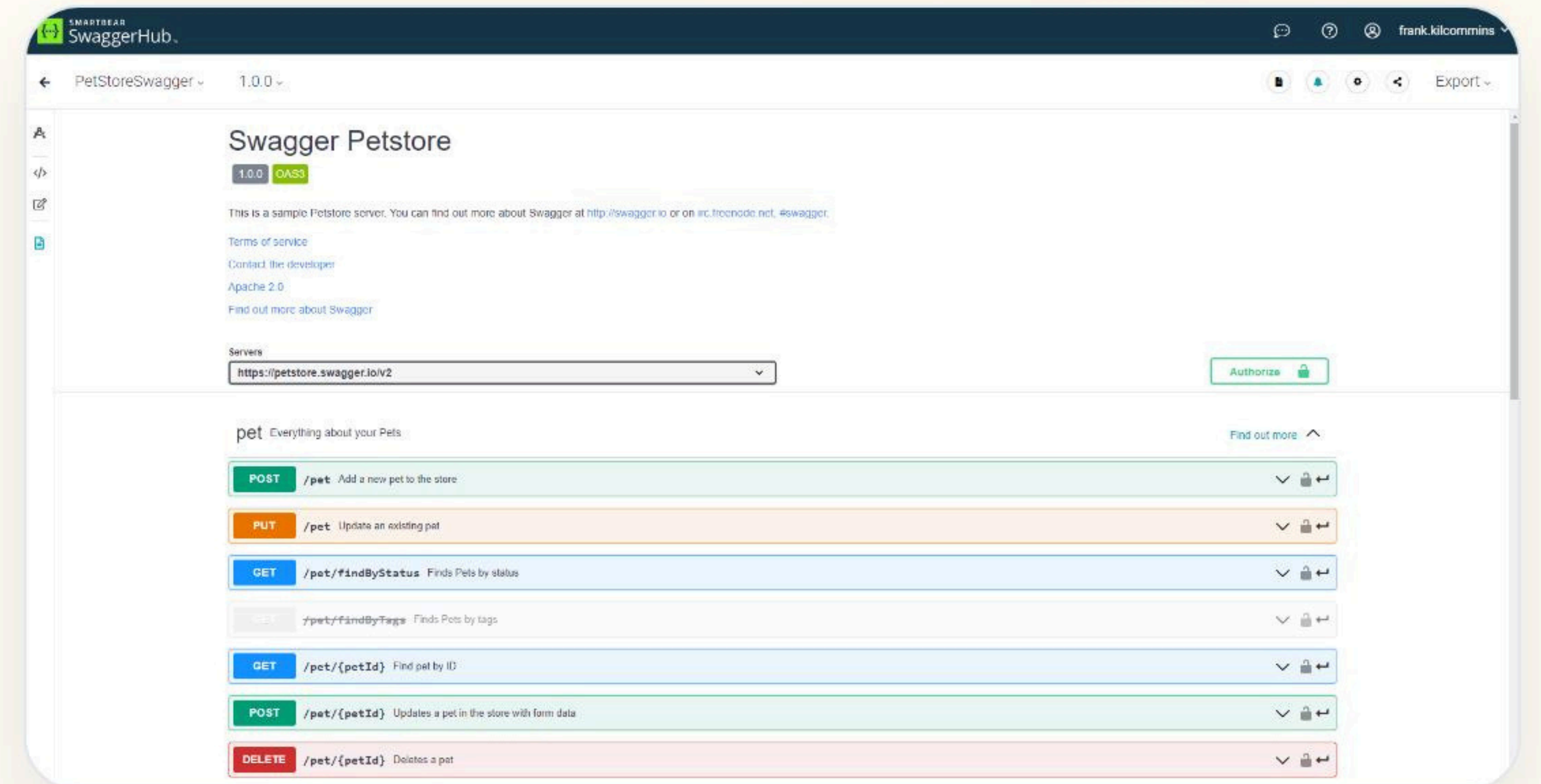
Logique CRUD

Métier regroupé dans le dossier app avec les routes.

Documentation Interactive

Nous avons intégré **Swagger** pour faciliter les tests et la correction. Accessible sur le port **8080**, il permet :

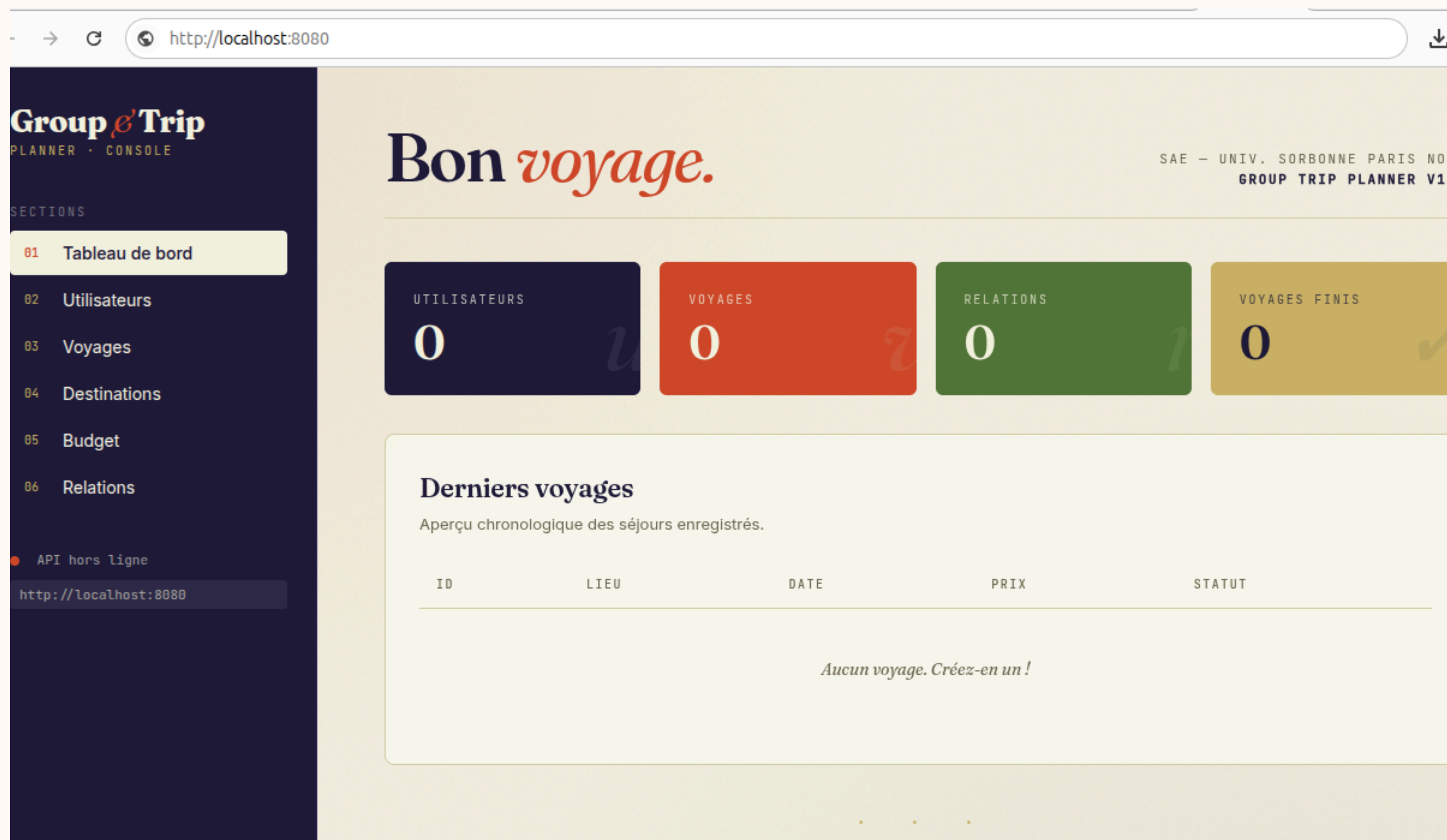
- ✎ Tester chaque endpoint en temps réel.
- ✓ Vérifier les contraintes de persistance.
- </> Visualiser les schémas de réponse.



27 Endpoints répartis en 5 modules

Module	Actions REST	Exemple Endpoint
Utilisateurs	Créer, Lister, Détails, Update, Delete	GET /utilisateurs/all/{email}
Voyages	Créer, Lister, Lieu, Prix, Statut, Update	GET /voyages/all/{voyage_id}
Relations	Lier Voyage-User, Lister, Filtres, Delete	POST /relations
Destinations	Ajouter étape, Lister, Détail, Update, Delete	POST /voyages/{id}/destinations
Budget	Créer, Récupérer, Mettre à jour	GET /voyages/{trip_id}/budget

Interface "Bon voyage."



Design épuré & réactif



Palette de couleurs équilibrée.



Interface sobre avec Flexbox.



Polices : Inter, Fraunces, JetBrains Mono.

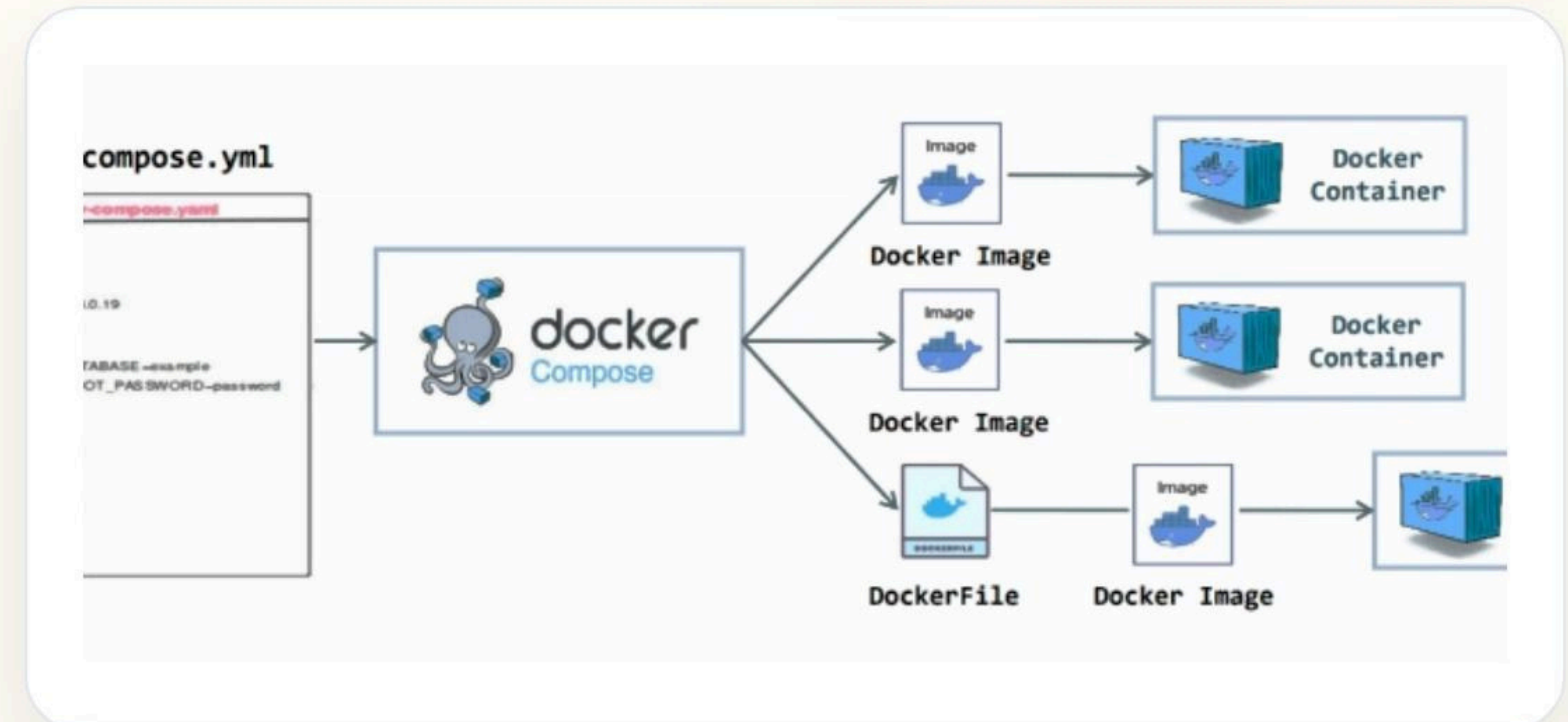
Statistiques en temps réel :

Visualisation directe des Utilisateurs, Voyages, Relations et Voyages Finis.

Déploiement Conteneurisé

Utilisation de **Docker Compose** pour orchestrer les services :

-  **Conteneur Backend** : Image Python optimisée.
-  **Conteneur DB** : PostgreSQL persistant.
-  **Auto-Build** : Paramètre `--build` pour intégrer les dernières modifications source.





Merci de votre attention

Avez-vous des questions ?

Zeynab SALEHI | Boboc TEODORA | Mathurin MARTEL

Group Trip Planner - v1.0